A/B Test: Credit Top-Up Reminder

Context

We are analysts on the onboarding team. The goal of the zone is to **increase conversion to first payment**.

Users communicate in the app using an internal currency — credits. Upon registration, each user receives **20 bonus credits**. When credits run out, a *"insufficient credits"* message appears — and this is where many users leave without making a payment.

Test hypothesis: if we remind the user to top up *before* the credits run out, they are more likely to make their first payment.

Test mechanic: a pinned message appears in the chat when ≤5 credits remain. It disappears after the first payment and is never shown again.

Hypotheses and Parameters

- **H₀:** Conversion to first payment is equal in both groups: $p_{control} = p_{test}$
- **H₁:** Conversion in the test group is higher: $p_{test} > p_{control}$

Parameter	Value
Significance level α	0.05
Power (1–β)	0.80
Test type	One-sided z-test for proportions
Split	50% / 50%

> **Why one-sided?** We launch the reminder with a clear hypothesis that it will increase conversion. The scenario "test worsened results" is not relevant for our decision — so a one-sided test is appropriate and gives higher power at the same α.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats
from statsmodels.stats.proportion import proportions_ztest, proportion_effectsize
from statsmodels.stats.power import NormalIndPower

# Load data
hist = pd.read_csv('ab_test_task_historical_data.csv')
test = pd.read_csv('ab_test_task_data.csv')

# Convert date columns
for col in ['date_reg', 'date_first_payment', 'date_spent_15_credits']:
    hist[col] = pd.to_datetime(hist[col], utc=True, errors='coerce')

for col in ['date_reg', 'date_first_payment', 'date_spent_15_credits', 'date_reminder']:
    test[col] = pd.to_datetime(test[col], utc=True, errors='coerce')

print('historical_data:', hist.shape)
print('test_data:      ', test.shape)
display(hist.head(3))
display(test.head(3))
```

```
historical_data: (59758, 4)
test_data:       (125217, 6)
```

	id_user	date_reg	date_first_payment	date_spent_15_credits
0	1	2023-03-01 00:00:09.068945+00:00	2023-03-01 02:06:31.472018+00:00	2023-03-01 02:00:38+00:00
1	2	2023-03-01 00:00:25.615471+00:00	NaT	2023-03-01 00:17:38+00:00
2	3	2023-03-01 00:00:39.084989+00:00	2023-03-01 00:52:37.812217+00:00	2023-03-01 00:18:53+00:00

	id_user	date_reg	date_first_payment	date_spent_15_credits	date_reminder	match
0	1	2023-03-14 08:37:01.855460+00:00	2023-03-14 11:25:23.962741+00:00	2023-03-14 09:15:31+00:00	2023-03-14 09:04:03.823970+00:00	1
1	2	2023-03-14 08:37:20.988023+00:00	NaT	NaT	NaT	0

What we have in the data:

- `historical_data` — 59,758 users before the test launch. Contains registration date, first payment date, and 15-credit spend date. Used to calculate baseline conversion and test duration.
- `test_data` — 125,217 users from the test launch. Additionally contains `date_reminder` (when the reminder was seen) and `match` (0 = control, 1 = test).

Task 1: How many days does the test need?

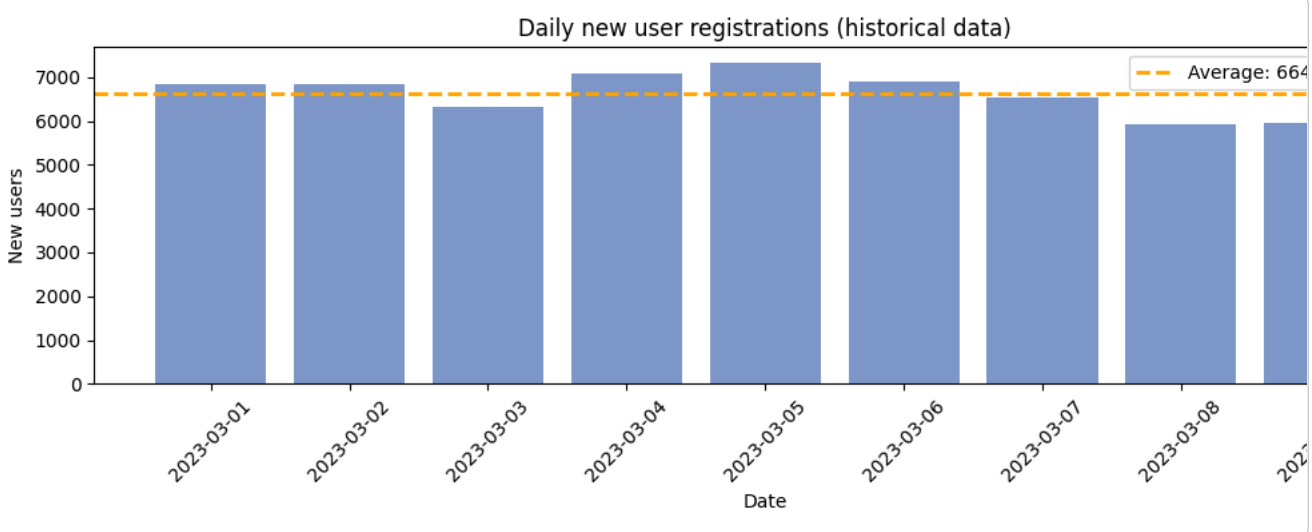
Before launching the test, we need to calculate the **minimum duration** for statistically reliable results. The logic:

1. Take baseline conversion from historical data — this is our reference point
2. Define MDE (Minimum Detectable Effect) — the minimum uplift that is practically meaningful. We choose 10% relative — i.e. if current CR = 5.45%, target = 5.99%. A smaller effect is either unrealistic to detect or not worth the development cost
3. Use power analysis to find the required number of users per group
4. Divide by daily new user inflow at 50/50 split → get number of days

```
# Baseline conversion from historical data
base_cr = hist['date_first_payment'].notna().mean()
print(f'Baseline conversion: {base_cr*100:.2f}%')

# Average daily new user inflow
avg_daily = hist.groupby(hist['date_reg'].dt.date)['id_user'].count().mean()
print(f'Average daily inflow: {avg_daily:.0f} users/day')

# Power analysis
p_control = base_cr
p_test    = base_cr * 1.10
effect_size = proportion_effectsize(p_control, p_test)

n_per_group = NormalIndPower().solve_power(
    effect_size=abs(effect_size),
    alpha=0.05,
    power=0.80,
    alternative='larger'
)
n_per_group = int(np.ceil(n_per_group))
days_needed = int(np.ceil(n_per_group / (avg_daily * 0.5)))

print(f'\nWith MDE=10% relative:')
print(f'  Required per group: {n_per_group:,} users')
print(f'  Required days:      {days_needed}')
```

```
Baseline conversion: 5.45%
Average daily inflow: 6640 users/day

With MDE=10% relative:
  Required per group: 22,446 users
  Required days:      7
```

```
# Chart: daily new user registrations
daily_reg = hist.groupby(hist['date_reg'].dt.date)['id_user'].count()

plt.figure(figsize=(11, 4))
plt.bar(daily_reg.index, daily_reg.values, color='#4870B6', alpha=0.7)
plt.axhline(avg_daily, color='orange', linestyle='--', linewidth=2,
            label=f'Average: {avg_daily:.0f} users/day')
plt.title('Daily new user registrations (historical data)')
plt.xlabel('Date')
plt.ylabel('New users')
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()
plt.show()
```



Metric	Value
Baseline conversion (historical)	**5.45%**
Chosen MDE	**10% relative** (5.45% → 5.99%)
Required users per group	**22,446**
Average daily new users	**6,640 users/day**
Required test duration	**~7 days**

With a steady registration flow (6,640 users/day) and 50/50 split, each group gains 3,320 users daily. To accumulate 22,446 users per group we need **7 days**.

Note: 7 days is the minimum to detect an effect ≥10%. If the real effect is smaller — the test would need to run longer or MDE should be revised.

```
# Actual test duration
test_start = test['date_reg'].min()
test_end   = test['date_reg'].max()
actual_days = (test_end - test_start).days

print(f'Test started:     {test_start.date()}')
print(f'Last user:        {test_end.date()}')
print(f'Actual duration:  {actual_days} days')
print(f'Minimum required: {days_needed} days')
print()
if actual_days >= days_needed:
    print(f'✅ Test ran long enough — results can be trusted.')
else:
    print(f'⚠️ Test ran shorter than required minimum — results are preliminary.')
```

```
Test started:     2023-03-14
Last user:        2023-04-18
Actual duration:  34 days
Minimum required: 7 days

✅ Test ran long enough — results can be trusted.
```

The actual test duration **exceeds the calculated 7-day minimum** — however this does not mean all 125K users can be included in the analysis. Users who registered in the last 7 days of the test did not have a full conversion window. Therefore we form an **eligible cohort** — excluding those who registered after the cutoff date.

```
# Eligible cohort — exclude users without a full conversion window
CONVERSION_WINDOW = 7  # days — taken from power analysis

test_end = test['date_reg'].max()
cutoff = test_end - pd.Timedelta(days=CONVERSION_WINDOW)

eligible = test[test['date_reg'] <= cutoff]

print(f'Total users:    {len(test):,}')
print(f'Eligible users: {len(eligible):,}')
print(f'Excluded:       {len(test) - len(eligible):,}')
print(f'Cutoff date:    {cutoff.date()}')
```

```
Total users:    125,217
Eligible users: 101,962
Excluded:       23,255
Cutoff date:    2023-04-11
```

Task 2: Evaluating Test Results

Step 1: SRM Check (Sample Ratio Mismatch)

Before analyzing metrics, we verify that users were correctly distributed across groups. If the split is not 50/50 — this signals a technical error and results cannot be trusted.

```
# SRM Check — on eligible cohort
n_ctrl = (eligible['match'] == 0).sum()
n_trt  = (eligible['match'] == 1).sum()
expected = (n_ctrl + n_trt) / 2

chi2, p_srm = stats.chisquare([n_ctrl, n_trt], f_exp=[expected, expected])

print(f'Control: {n_ctrl:,}  |  Test: {n_trt:,}  |  Expected: {expected:,.0f}')
print(f'p-value SRM: {p_srm:.4f}')
```

```
Control: 50,939  |  Test: 51,023  |  Expected: 50,981
p-value SRM: 0.7925
```

Control: 50,939 | Test: 51,023 | p-value = 0.7925 ≥ 0.05

No SRM detected. The split on the eligible cohort is nearly perfect (49.96% / 50.04%) — the test is set up correctly, proceeding with analysis.

Compared to the full dataset (62,568 / 62,649), the eligible cohort contains 101,962 users — 23,255 users without a full conversion window were excluded.

Step 2: Investigating Contaminated Control Users

```
ctrl_raw  = eligible[eligible['match'] == 0]
trt       = eligible[eligible['match'] == 1]

contaminated = ctrl_raw[ctrl_raw['date_reminder'].notna()]
ctrl_clean   = ctrl_raw[ctrl_raw['date_reminder'].isna()]

print(f'Contaminated in control: {len(contaminated)}')
print()

# 1. Distribution by date — uniform or spike?
print('Distribution of contaminated users by reminder date:')
print(contaminated['date_reminder'].dt.date.value_counts().sort_index())
print()

# 2. Is their conversion higher than clean control?
cr_contaminated = contaminated['date_first_payment'].notna().mean()
cr_clean_ctrl   = ctrl_clean['date_first_payment'].notna().mean()
print(f'CR contaminated:  {cr_contaminated:.2%}')
print(f'CR clean control: {cr_clean_ctrl:.2%}')
print()

# 3. Exclude from control
ctrl = ctrl_clean
```

```
print(f'Control after cleaning: {len(ctrl):,} users')
print(f'n_ctrl updated: {len(ctrl):,}')
n_ctrl = len(ctrl)
```

```
Contaminated in control: 89

Distribution of contaminated users by reminder date:
date_reminder
2023-03-14    88
2023-03-15     1
Name: count, dtype: int64

CR contaminated:  8.99%
CR clean control: 5.59%

Control after cleaning: 50,850 users
n_ctrl updated: 50,850
```

89 users in the control group have a `date_reminder` — this should not be possible. We investigate: when they received the reminder, whether their conversion is higher, and exclude them from the analysis.

⚠ **Validity risk:** CR of contaminated users (8.99%) is almost twice the CR of clean control (5.59%). This confirms that 89 users did not end up in control with a reminder by chance — they were artificially inflating control CR and suppressing the difference between groups. We exclude them and use `ctrl_clean` as the control group for further analysis.

Date distribution: 88 out of 89 received the reminder on the first day of the test (2023-03-14) — a signal of an implementation error at launch, not a systemic bug.

Step 3: Primary Metric — Conversion to First Payment

The primary test metric is the share of users who made their first payment (`date_first_payment` is not null). We compare between groups using a one-sided z-test.

```
x_ctrl  = ctrl['date_first_payment'].notna().sum()
x_trt   = trt['date_first_payment'].notna().sum()
cr_ctrl = x_ctrl / n_ctrl
cr_trt  = x_trt  / n_trt

print(f'Control CR: {x_ctrl}/{n_ctrl} = {cr_ctrl*100:.2f}%')
print(f'Test CR:    {x_trt}/{n_trt} = {cr_trt*100:.2f}%')
print(f'Abs. difference: {(cr_trt-cr_ctrl)*100:+.2f} p.p.')
print(f'Relative lift:   {(cr_trt-cr_ctrl)/cr_ctrl*100:+.1f}%')

# Z-test
z_stat, p_val = proportions_ztest([x_trt, x_ctrl], [n_trt, n_ctrl], alternative='larger')

# Confidence interval
diff = cr_trt - cr_ctrl
se   = np.sqrt(cr_ctrl*(1-cr_ctrl)/n_ctrl + cr_trt*(1-cr_trt)/n_trt)
ci   = (diff - 1.96*se, diff + 1.96*se)

print(f'\nz-stat = {z_stat:.4f},  p-value = {p_val:.4f}')
print(f'95% CI: [{ci[0]*100:.2f}%, {ci[1]*100:.2f}%]')
print()
print('Reject H₀ (difference is significant)' if p_val < 0.05 else 'Fail to reject H₀ (difference not significant)')
```

```
Control CR: 2840/50850 = 5.59%
Test CR:    3020/51023 = 5.92%
Abs. difference: +0.33 p.p.
Relative lift:   +6.0%

z-stat = 2.2882,  p-value = 0.0111
95% CI: [0.05%, 0.62%]

Reject H₀ (difference is significant)
```

Z-test results (eligible cohort, clean control):

	Control	Test
Group size	50,850	51,023
Number of payments	2,840	3,020
CR	**5.59%**	**5.92%**

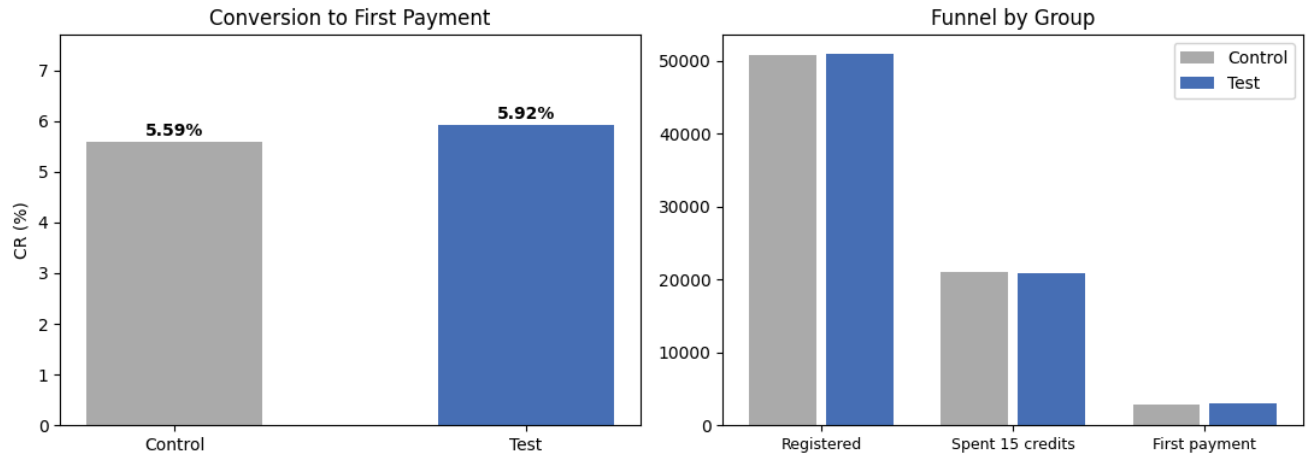
	Control	Test
Absolute difference		**+0.33 p.p.**
Relative lift		**+6.0%**
p-value		**0.0111**
95% CI of difference		**[+0.05%, +0.62%]**

p-value = 0.0111 < 0.05 — we reject H_0. The difference is statistically significant.

The confidence interval **does not include 0** — the result is not random. The reminder produced a +6.0% relative lift in conversion.

The number of registered users and those who spent 15 credits is nearly identical across groups — expected, since the reminder does not affect activity before the ≤5 credit threshold is reached. The difference appears specifically at the first payment step.

```
# Conversion chart and funnel
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

ax = axes[0]
bars = ax.bar(['Control', 'Test'], [cr_ctrl*100, cr_trt*100],
              color=['#AAAAAA', '#4870B6'], width=0.5)
for bar, val in zip(bars, [cr_ctrl*100, cr_trt*100]):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.05,
            f'{val:.2f}%', ha='center', va='bottom', fontweight='bold')
ax.set_title('Conversion to First Payment')
ax.set_ylabel('CR (%)')
ax.set_ylim(0, max(cr_ctrl, cr_trt)*100 * 1.3)

ax2 = axes[1]
steps = ['Registered', 'Spent 15 credits', 'First payment']
ctrl_vals = [n_ctrl, ctrl['date_spent_15_credits'].notna().sum(), x_ctrl]
trt_vals  = [n_trt,  trt['date_spent_15_credits'].notna().sum(),  x_trt]
x = np.arange(len(steps))
ax2.bar(x - 0.2, ctrl_vals, width=0.35, label='Control', color='#AAAAAA')
ax2.bar(x + 0.2, trt_vals,  width=0.35, label='Test',    color='#4870B6')
ax2.set_xticks(x)
ax2.set_xticklabels(steps, fontsize=9)
ax2.set_title('Funnel by Group')
ax2.legend()

plt.tight_layout()
plt.show()
```



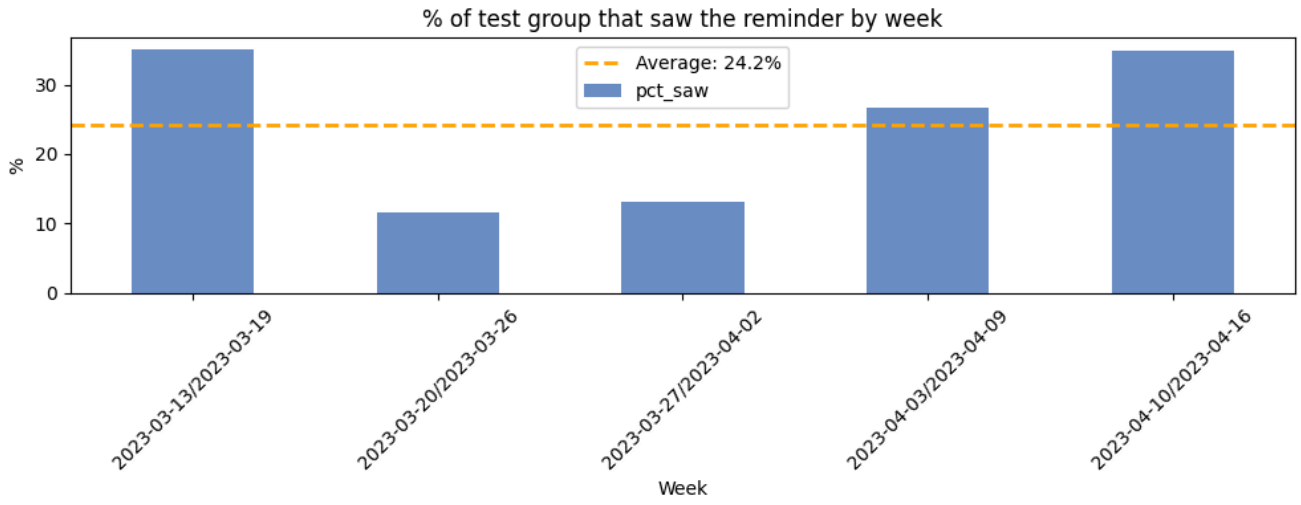
Step 4: Reminder Display Stability by Week

We check whether the reminder was shown consistently throughout the test. A sharp spike or drop is a signal of a bug in the implementation.

```
# Reminder display stability by week
trt_copy = trt.copy()
trt_copy['week'] = trt_copy['date_reg'].dt.to_period('W')

weekly = trt_copy.groupby('week').agg(
    total=('id_user', 'count'),
```

```
    saw_reminder=('date_reminder', lambda x: x.notna().sum())
).assign(pct_saw=lambda df: df['saw_reminder'] / df['total'] * 100)

print('Reminder display by week:')
print(weekly[['total', 'saw_reminder', 'pct_saw']].round(1).to_string())

fig, ax = plt.subplots(figsize=(10, 4))
weekly['pct_saw'].plot(kind='bar', ax=ax, color='#4870B6', alpha=0.8)
ax.axhline(weekly['pct_saw'].mean(), color='orange', linestyle='--', linewidth=2,
           label=f'Average: {weekly["pct_saw"].mean():.1f}%')
ax.set_title('% of test group that saw the reminder by week')
ax.set_ylabel('%')
ax.set_xlabel('Week')
ax.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

print(f'\nMin: {weekly["pct_saw"].min():.1f}%  |  Max: {weekly["pct_saw"].max():.1f}%  |  Diff: {weekly["pct_saw"].max()-weekly[
print('✅ Display stable' if weekly['pct_saw'].max() - weekly['pct_saw'].min() < 5 else '⚠️ Notable instability — check impleme
```

```
Reminder display by week:
                      total  saw_reminder  pct_saw
week
2023-03-13/2023-03-19  11169          3910     35.0
2023-03-20/2023-03-26  13165          1517     11.5
2023-03-27/2023-04-02  12575          1638     13.0
2023-04-03/2023-04-09  11802          3152     26.7
2023-04-10/2023-04-16   2312           807     34.9
/tmp/ipykernel_7463/476695834.py:3: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
  trt_copy['week'] = trt_copy['date_reg'].dt.to_period('W')
```




```
Min: 11.5%  |  Max: 35.0%  |  Diff: 23.5 p.p.
⚠️ Notable instability — check implementation logs
```

```
# Check: was the audience consistent across weeks?
# If pct_active is stable — display instability is an implementation bug, not traffic quality
weekly_activity = trt_copy.groupby('week').agg(
    total=('id_user', 'count'),
    spent_credits=('date_spent_15_credits', lambda x: x.notna().sum())
).assign(pct_active=lambda df: df['spent_credits'] / df['total'] * 100)

print('Audience activity by week (% who spent 15 credits):')
print(weekly_activity[['total', 'spent_credits', 'pct_active']].round(1).to_string())
```

```
Audience activity by week (% who spent 15 credits):
                      total  spent_credits  pct_active
week
2023-03-13/2023-03-19  11169           4658        41.7
2023-03-20/2023-03-26  13165           5320        40.4
2023-03-27/2023-04-02  12575           5146        40.9
2023-04-03/2023-04-09  11802           4871        41.3
2023-04-10/2023-04-16   2312            943        40.8
```

Root cause check:

`pct_active` (~41% across all weeks without exception) — the audience is consistent and traffic quality is stable.

This means the reminder display instability (11.5% → 35%) is **not explained** by audience quality. The cause is the reminder display implementation. During weeks 20.03–02.04 the reminder mechanic was not working correctly.

Implication: the measured lift of +6.0% is an underestimate of the true effect — had the reminder been shown consistently across all 5 weeks, the lift could have been higher. We recommend checking display logs for 20.03–02.04 before full rollout.

Step 5: Reminder Analysis (date_reminder)

We verify the implementation quality of the test and examine conversion among those who actually saw the reminder.

```
# Reminder analysis — on eligible cohort
saw     = trt[trt['date_reminder'].notna()]
not_saw = trt[trt['date_reminder'].isna()]

print(f'Test group — saw reminder:     {len(saw):,} ({len(saw)/n_trt*100:.1f}%),  CR={saw["date_first_payment"].notna().mean()*1
print(f'Test group — did NOT see reminder: {len(not_saw):,} ({len(not_saw)/n_trt*100:.1f}%),  CR={not_saw["date_first_payment"].
print(f'Control — saw reminder (should be 0): {ctrl["date_reminder"].notna().sum()} (excluded)')
```

```
Test group — saw reminder:     11,024 (21.6%),  CR=16.56%
Test group — did NOT see reminder: 39,999 (78.4%),  CR=2.99%
Control — saw reminder (should be 0): 0 (excluded)
```

Reminder analysis results (eligible cohort):

Segment	Count	CR
Test — saw reminder	11,024 (21.6%)	**16.56%**
Test — did NOT see reminder	39,999 (78.4%)	**2.99%**
Control — saw reminder	0 (excluded)	—

1. Only 21.6% of the test group saw the reminder. This is expected — the reminder appears only when ≤5 credits remain, i.e. only for active users who spent most of their bonus balance. 78.4% either hadn't reached this threshold or were inactive.
2. CR among those who saw the reminder — 16.56% vs 2.99% among those who didn't. The reminder effectively converts exactly the audience it is shown to.

> **Important note on interpretation:** the 16.56% vs 2.99% difference is **not** the effect of the reminder. This is selection bias: users who saw the reminder are inherently more active (they spent ≥15 credits) and would have converted better even without it. The true effect of the reminder is measured by the z-test between groups (+6.0%, Step 3).

3. **Next test question:** why did 78.4% of the test group never see the reminder? This signals drop-off earlier in the funnel — there is also potential for improvement there.

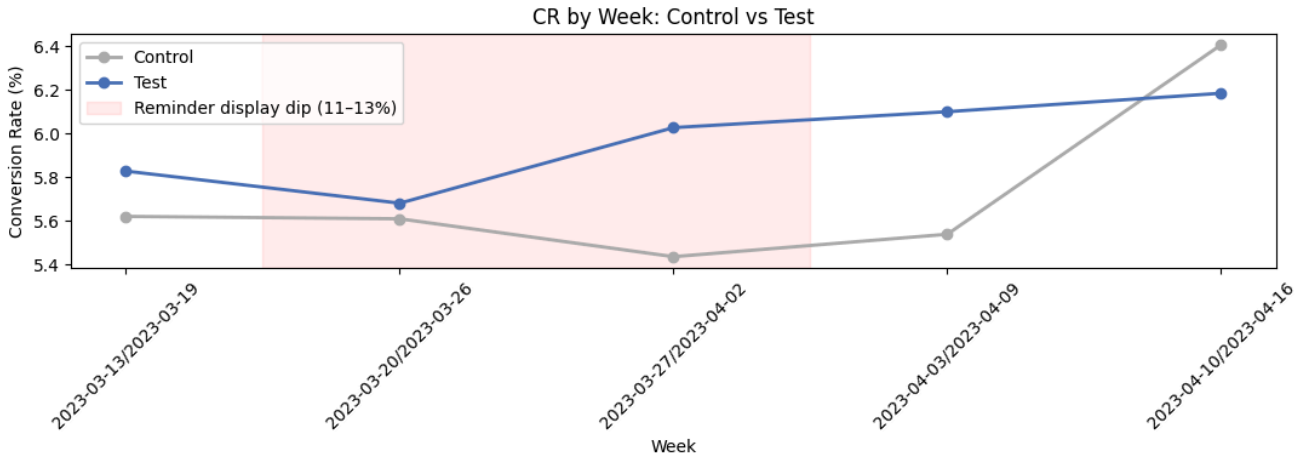
Step 6: Weekly CR Dynamics

We check whether the effect appeared immediately or grew over time, and whether a dip is visible during the problematic weeks (20.03–02.04) when the reminder was shown to only 11–13% of the test group.

```
# CR by week for control and test
eligible_copy = eligible.copy()
eligible_copy['week'] = eligible_copy['date_reg'].dt.to_period('W').astype(str)

weekly_cr = eligible_copy.groupby(['week', 'match']).agg(
    total=('id_user', 'count'),
    paid=('date_first_payment', lambda x: x.notna().sum())
).assign(cr=lambda df: df['paid'] / df['total'] * 100).reset_index()

fig, ax = plt.subplots(figsize=(11, 4))

for match_val, label, color in [(0, 'Control', '#AAAAAA'), (1, 'Test', '#4870B6')]:
    d = weekly_cr[weekly_cr['match'] == match_val]
    ax.plot(d['week'], d['cr'], label=label, color=color, marker='o', linewidth=2)

# We mark problem weeks (indexes 1 and 2 - March 20-26 and March 27-April 2)
ax.axvspan(0.5, 2.5, alpha=0.08, color='red', label='Reminder display dip (11–13%)')

ax.set_title('CR by Week: Control vs Test')
ax.set_ylabel('Conversion Rate (%)')
ax.set_xlabel('Week')
```

```
ax.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_7463/2650456529.py:3: UserWarning: Converting to PeriodArray/Index representation will drop timezone information.
  eligible_copy['week'] = eligible_copy['date_reg'].dt.to_period('W').astype(str)
```



Final Conclusion and Recommendation

Why roll out?

The test showed a clear positive result on clean data (eligible cohort, contaminated control users excluded).

p-value = 0.0111 — the probability of a random result is below 1.1%, the effect is real.

In absolute terms the uplift is +0.33 p.p. With ~6,600 new users per day, this is approximately **22 additional paying users per day**, or roughly **660 people per month** who would otherwise have left without paying.

The mechanic works precisely: among those who actually saw the reminder, CR = 16.56% vs 2.99% among those who didn't — the reminder appears at the right moment and does not disturb users who are not yet ready to pay.

Caveats before full rollout:

- Reminder display instability detected during weeks 20.03–02.04 (11.5% vs 35% in other weeks). The cause is implementation, not traffic quality. The real effect may be higher than +6.0% after the fix.
- Recommend checking display logs for the problematic period before full rollout.

Next Steps

1. Investigate reminder display logs for 20.03–02.04 — identify the root cause of the dip
2. After fixing, roll out to all users and monitor CR for 1–2 more weeks
3. Next hypothesis: why did 78.4% of the test group never reach the reminder? There is funnel drop-off earlier — potential for further improvement there